

Designing and Analysis of Algorithms

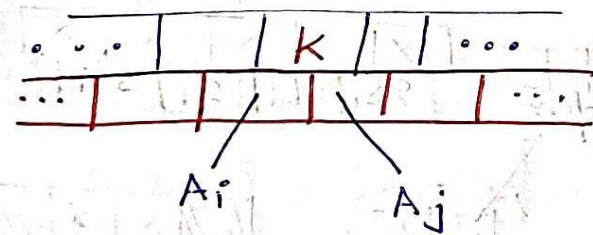
Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 18**

Deletion in an m-way Search Tree →

- Let K be a key to be deleted from the m-way search tree.
- To delete the key we proceed as one would to search for the key.



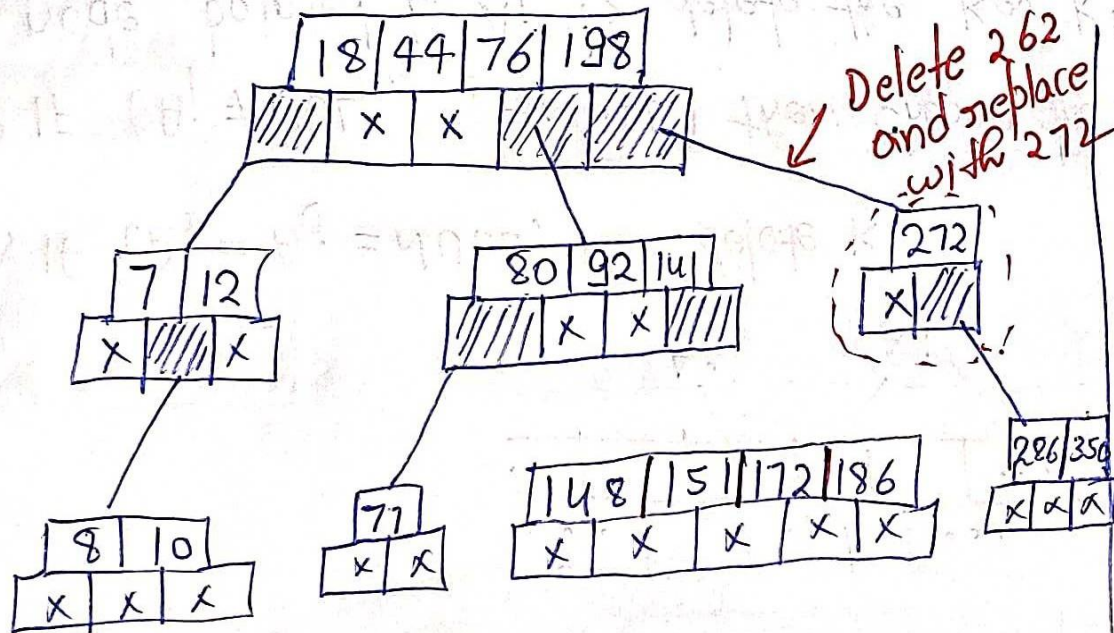
K : key
 A_i, A_j : Pointers to subtrees

- 1) IF $(A_i = A_j = \text{NULL})$ then delete K
- 2) IF $(A_i \neq \text{NULL}, A_j = \text{NULL})$ then choose the largest of the key element K' in the child node pointed by A_j
 - i) delete the key K'
 - ii) Replace K by K'
 - iii) Obviously deletion of K' may call for subsequent replacements and therefore deletions in a similar manner.

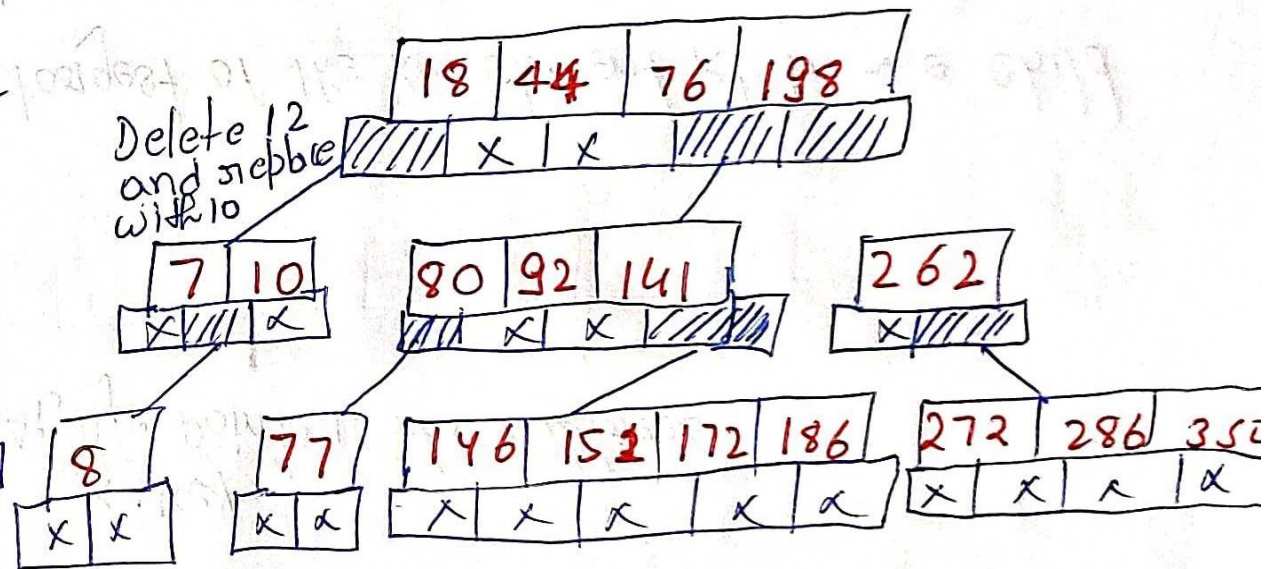
3) IF ($A_i = \text{NULL}$, $A_j \neq \text{NULL}$) then choose the smallest of the key element (k'') from the subtree pointed to by A_j delete k'' and replace k by k'' .

Again there may be some subsequent calls.

4) IF ($A_i \neq \text{NULL}$ and $A_j \neq \text{NULL}$) then choose either of 3 or 4.



(a)

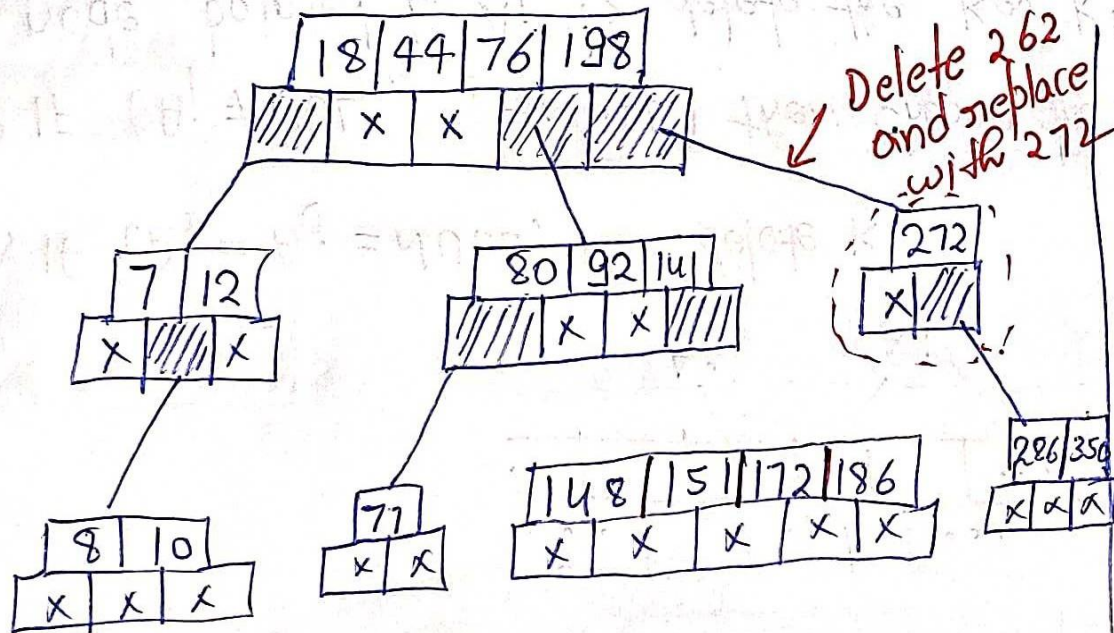


(b)

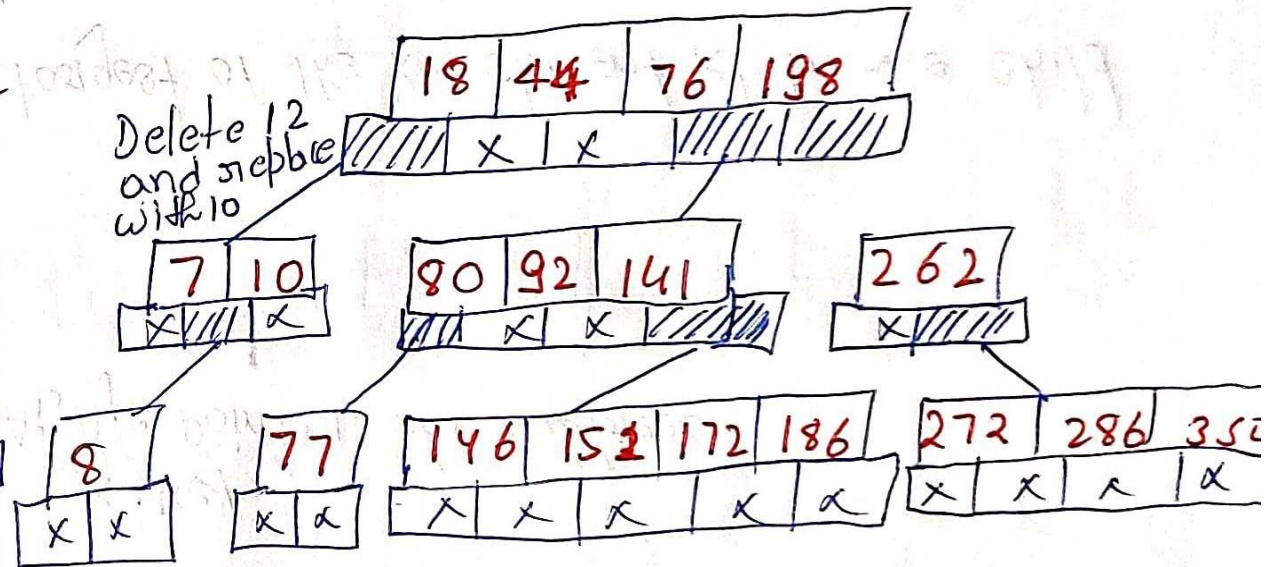
3) IF ($A_i = \text{NULL}$, $A_j \neq \text{NULL}$) then choose the smallest of the key element (k'') from the subtree pointed to by A_j delete k'' and replace k by k'' .

Again there may be some subsequent calls.

4) IF ($A_i \neq \text{NULL}$ and $A_j \neq \text{NULL}$) then choose either of 3 or 4.



(a)



(b)

B-Trees :

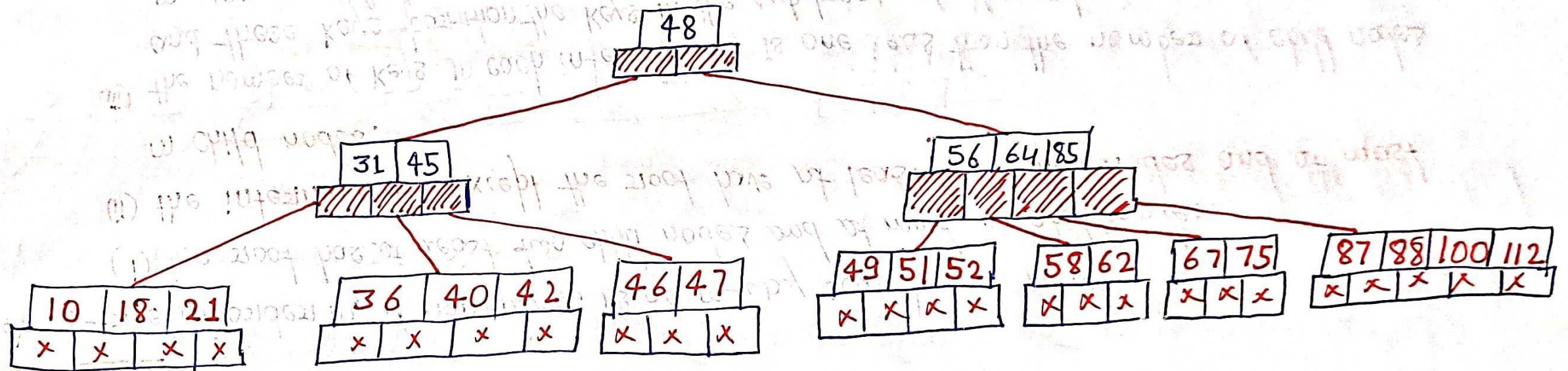
- > m-way search trees have the advantage of minimizing file access due to their restricted height.
- > However, it is essential that the height of the tree be kept as low as possible and therefore there arises the need to maintain balanced m-way trees. Such a balanced m-way search tree is what is defined as B-tree.

Definition →

A B-tree of order m , if non empty, is an m-way search tree in which:

- (i) the root has at least two child nodes and at most m child nodes.
- (ii) the internal nodes except the root have at least $\lceil \frac{m}{2} \rceil$ child nodes and at most m child nodes.
- (iii) the number of keys in each internal node is one less than the number of child nodes and these keys partition the keys in the subtrees of the node in a manner similar to that of m-way search trees.
- (iv) all the leaf nodes are on the same level.

* A B-tree of order 3 is referred to as 2-3 tree since the internal nodes are of degree 2 or 3 only.



Pointers to subtrees

Key fields

Null pointers

Searching a B-tree →

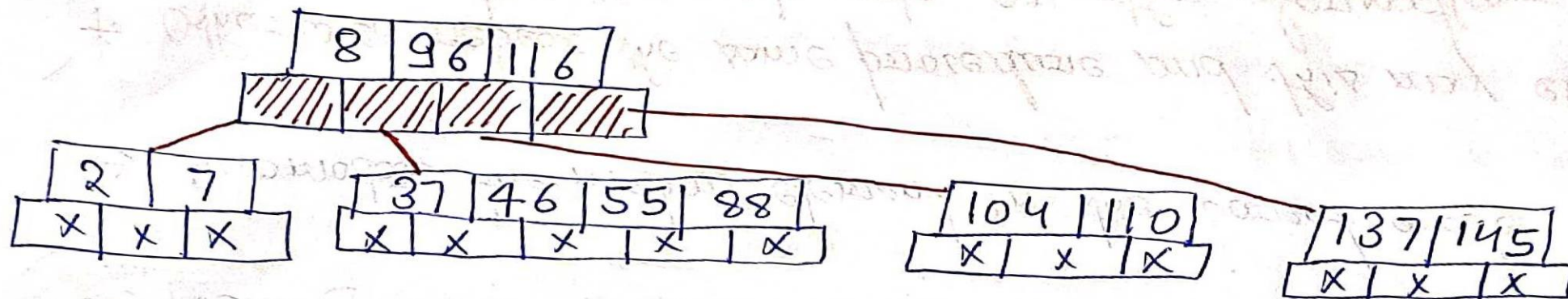
→ Searching for a key in a B-tree is similar to have an m-way search tree.

→ The number of access depends on the height h of the B-tree.

Insertion in a B-tree:

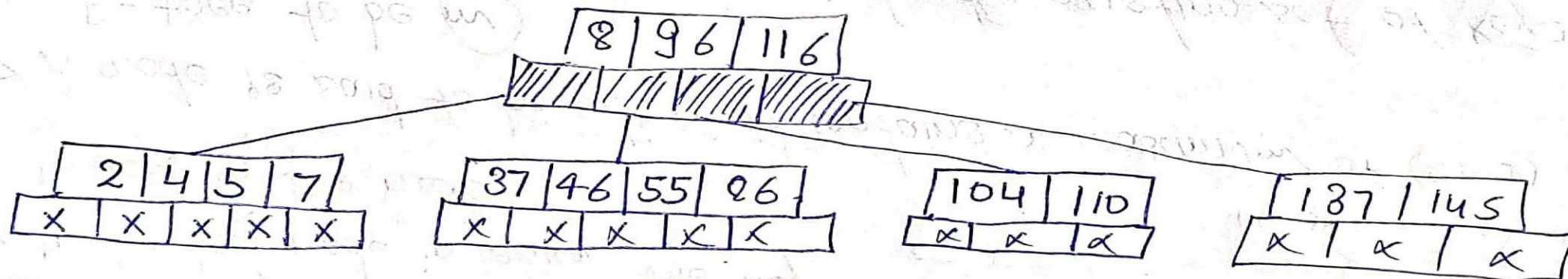
- > The insertion of a key in a B-tree proceeds as if one were searching for the key in the tree.
- > If the leaf node in which the key is to be inserted is not full, then the insertion is done in the node.
- > A node is said to be full if it contains a maximum of $(m-1)$ keys, given the order of B-tree to be m .
 - ↳ Insert the key in order into the existing set of keys in the node, split the node at its median into two nodes at the same level.
 - ↳ pushing the median element up by one level
 - ↳ Accomodate the median element in the parent node if it is not full
- * Otherwise repeat the same procedure and this may even call for rearrangement of the keys in the root node on the formation of a new root itself.

Consider the B-tree ÷

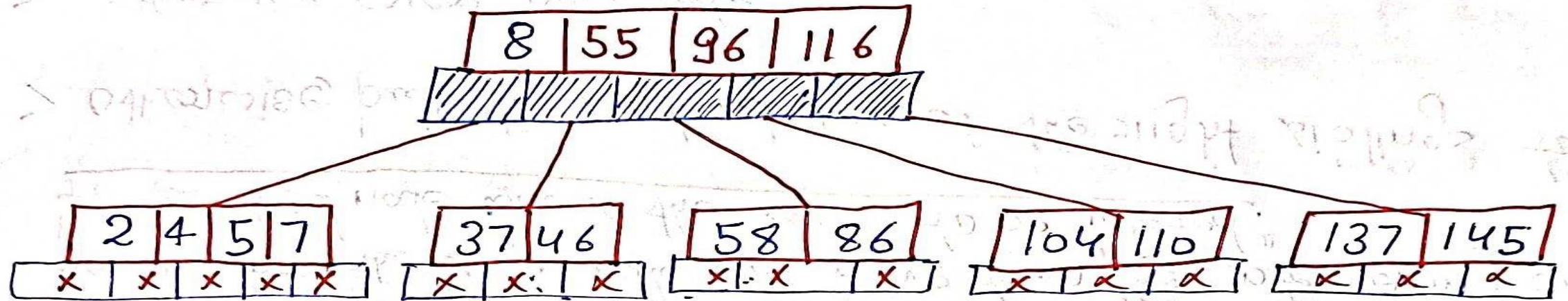


a) Insert the element 4, 5, 58, 6

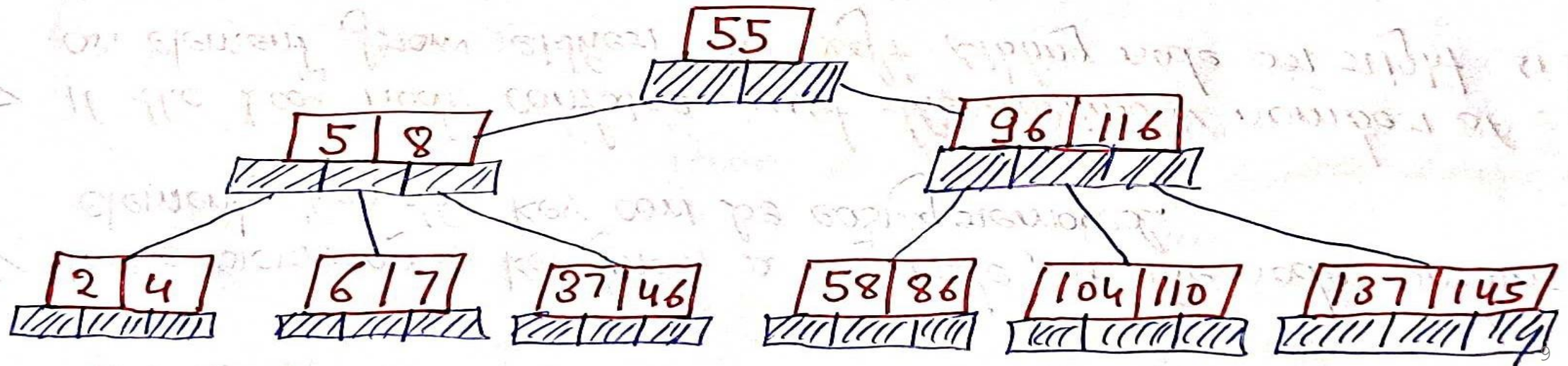
b) Insert 4, 5 (simpler)



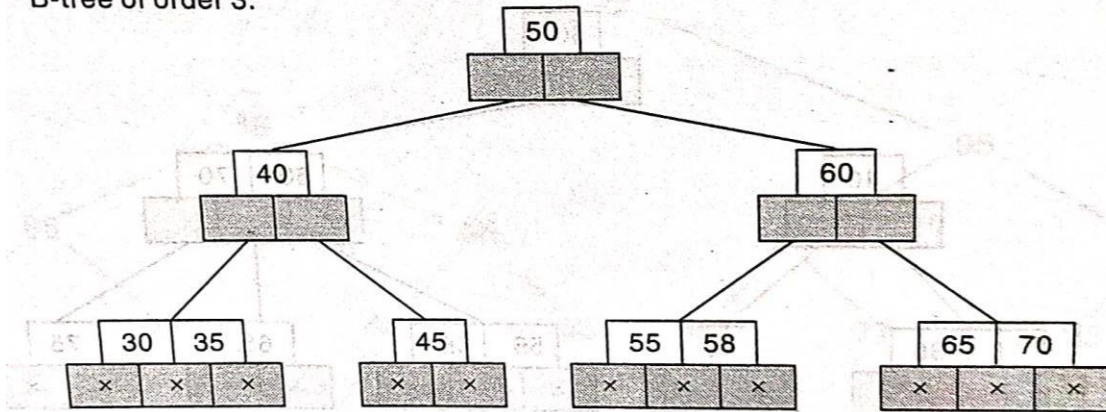
c) Insert 58



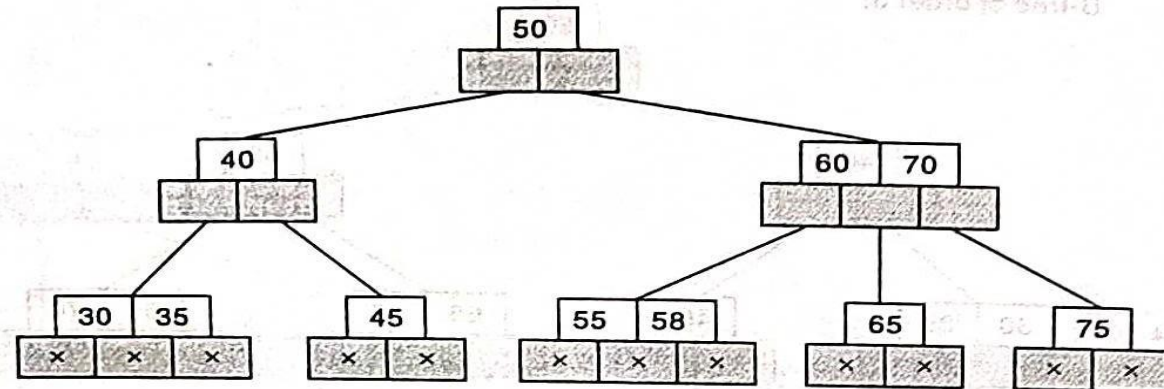
d) Insert 6:



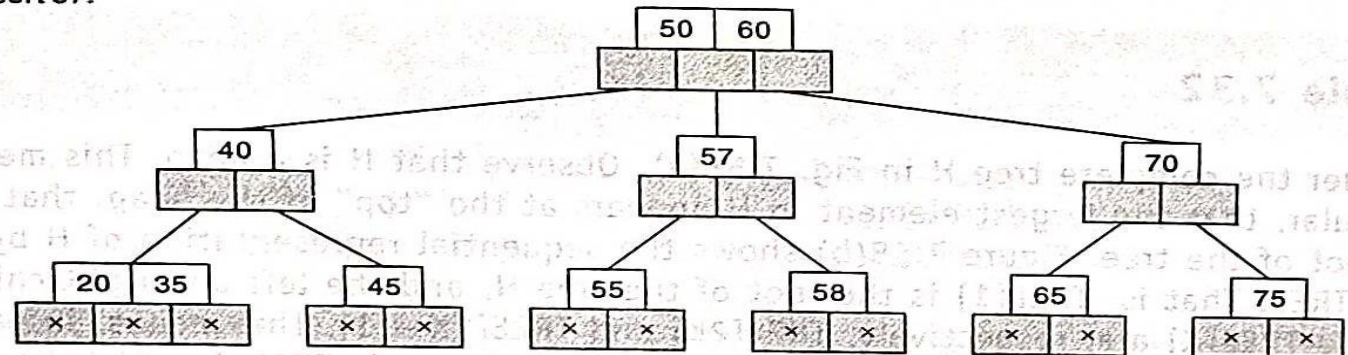
B-tree of order 3:



Insert 75:



Insert 57:



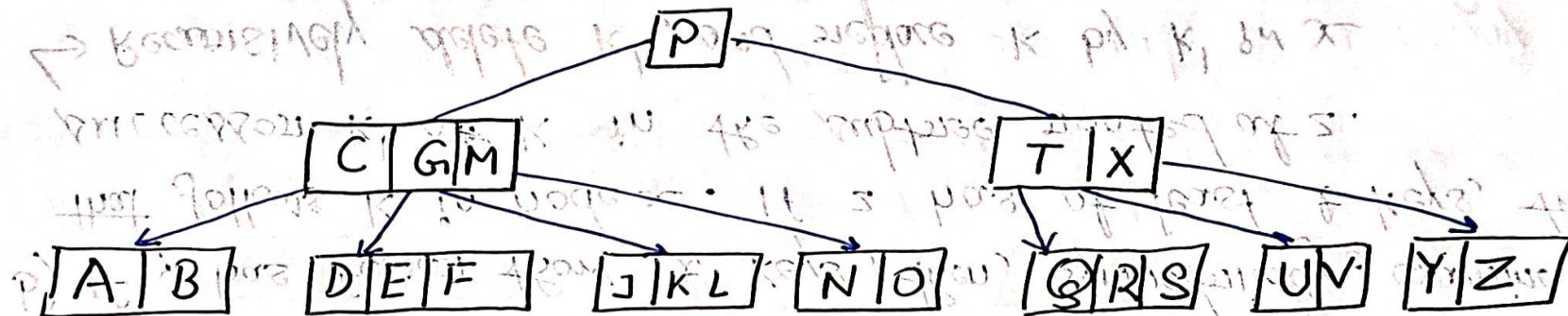
Deletion in a B-tree:

- > While deleting a key it is desirable that the keys in the leaf node are removed.
- > When a case arises forcing a key that is in an internal node to be deleted, then we promote a successor or a predecessor of the key to be deleted.
- > While removing a key from a leaf node, if the node contains more than the minimum node element, then the key can be easily removed.
- > If the leaf node contains just the minimum number of elements, then search from an element from either the left sibling node or right sibling node to fill the vacancy.
- > If the left sibling node has less than the minimum ~~node~~ number of keys, pull the largest key up into the parent node and move down the intervening entry "from the parent node to the leaf node where the key is to be deleted."
- > Otherwise, pull the smallest key of the right siblings to the left node.
- > otherwise create new node.

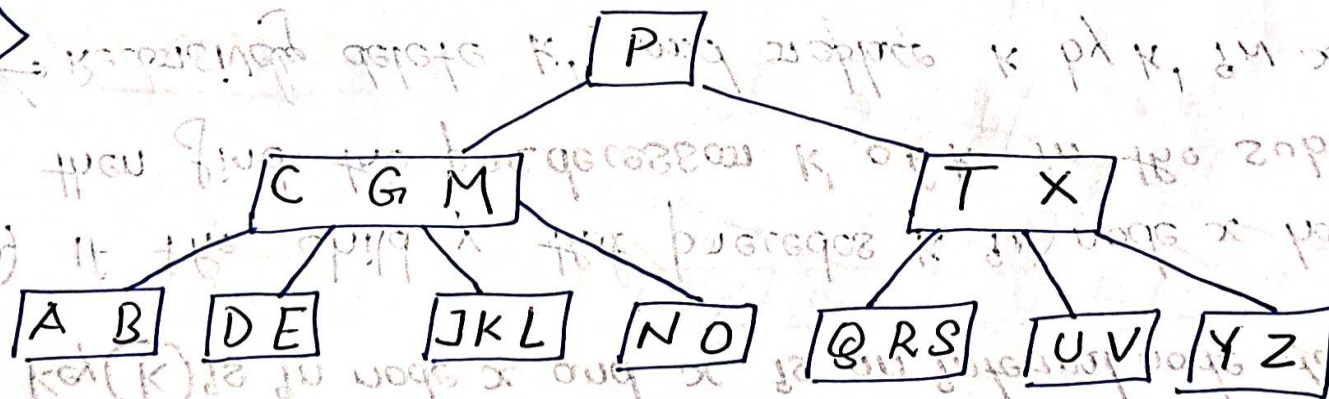
Deletion in B-Tree :-

Case: ① If the key k in node x and x is a leaf, delete the key k from x

①



<Delete F>



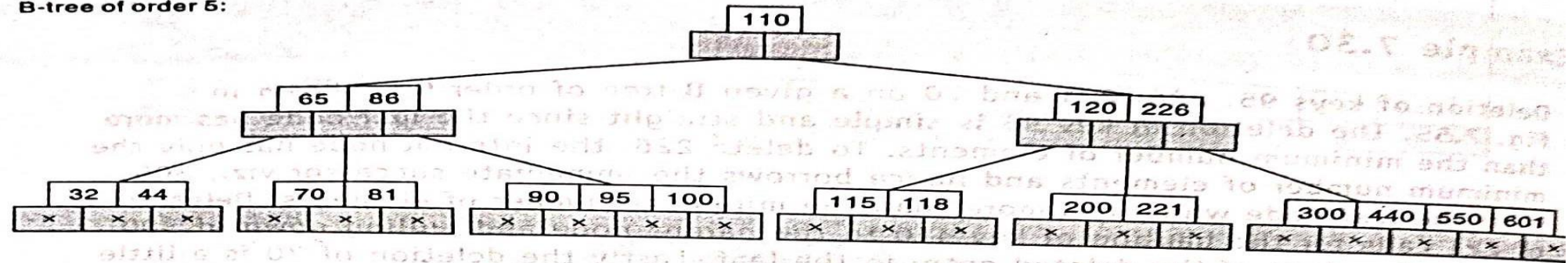
3a) If $x.c_i$ has only $t-1$ keys but has an immediate sibling with at least t keys give $x.c_i$ an extra key by moving a key from x down into $x.c_i$, moving a key $x.c_i$'s immediate left or right sibling up into x , and moving the appropriate child pointers from the sibling into $x.c_i$.

3b) If $x.c_i$ and both $x.c_i$'s immediate siblings have $t-1$ keys, merge $x.c_i$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

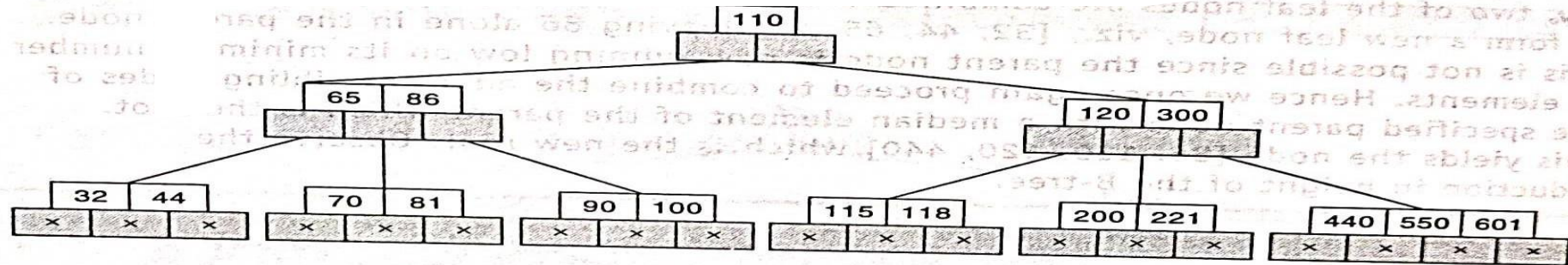


B deleted: case 3a

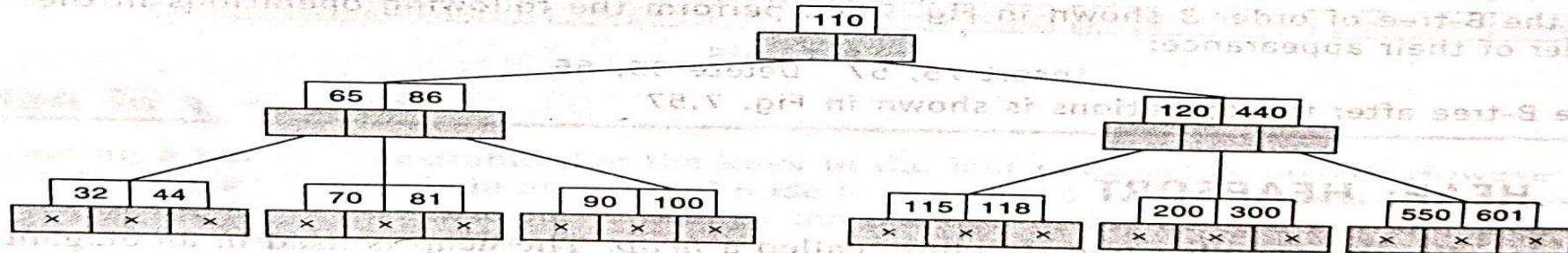
B-tree of order 5:



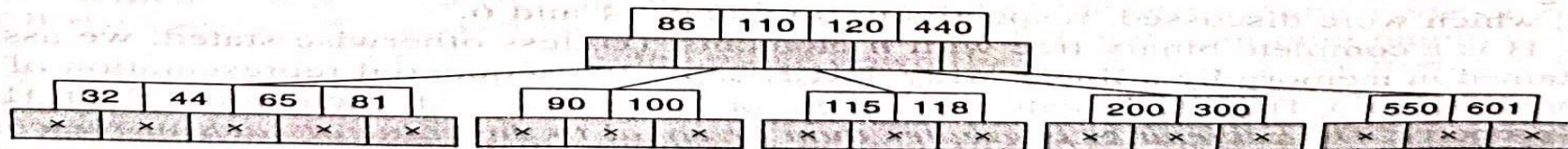
Delete 95, 226:



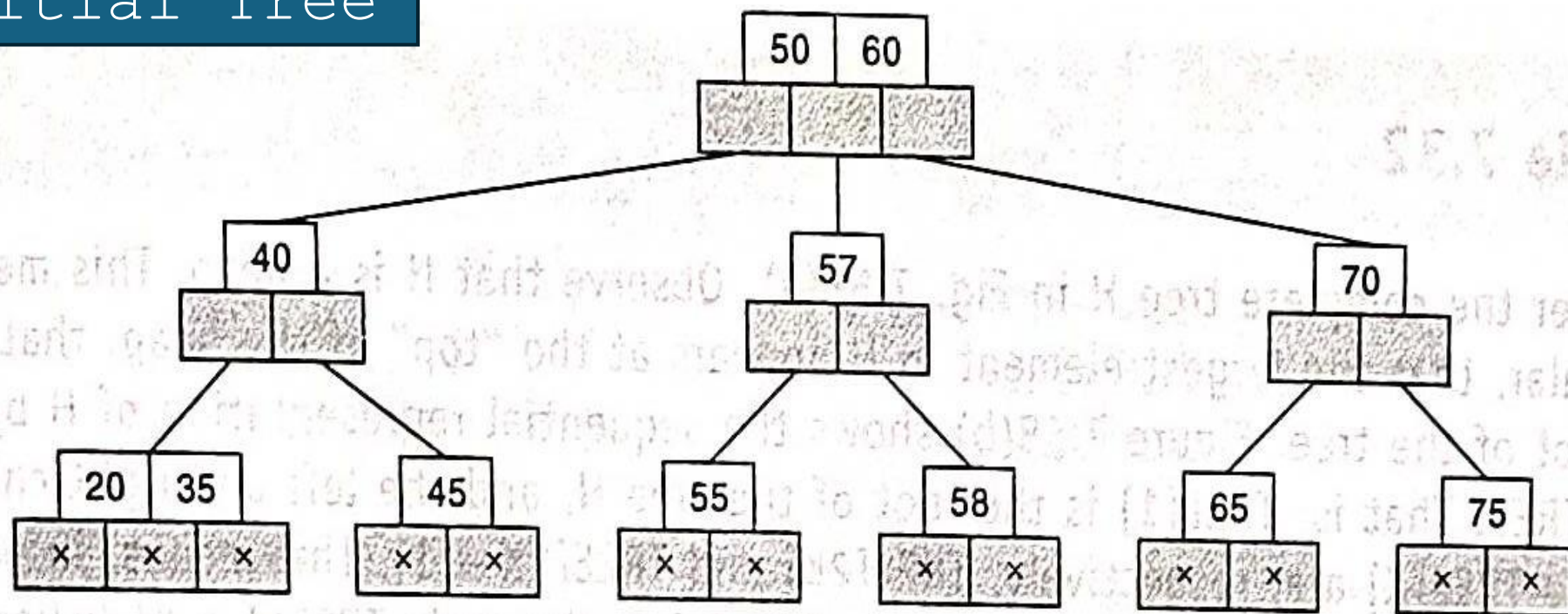
Delete 221:



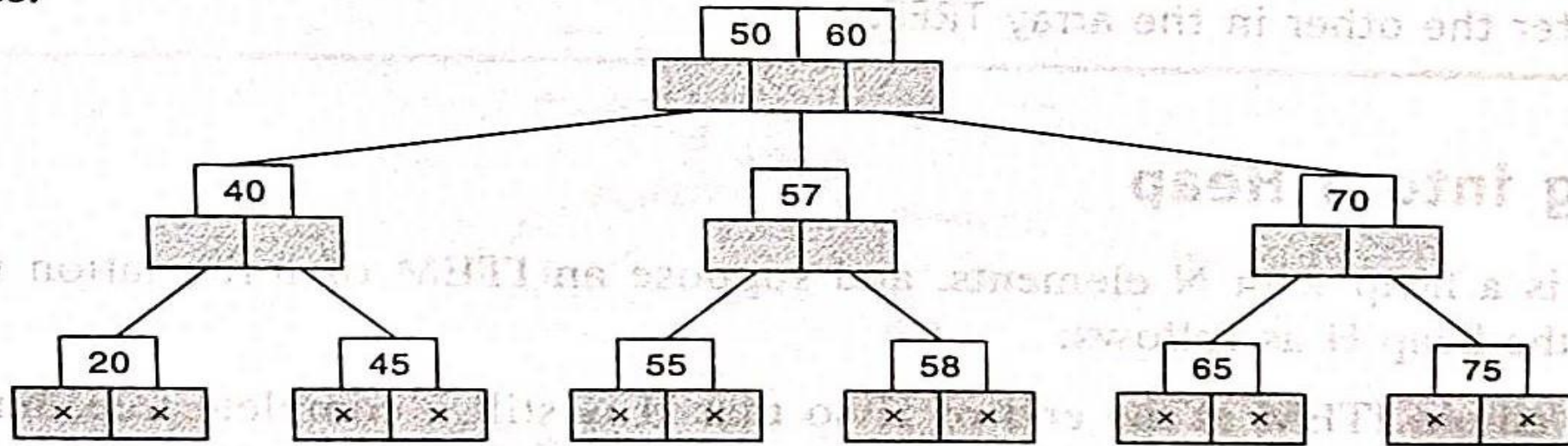
Delete 70:



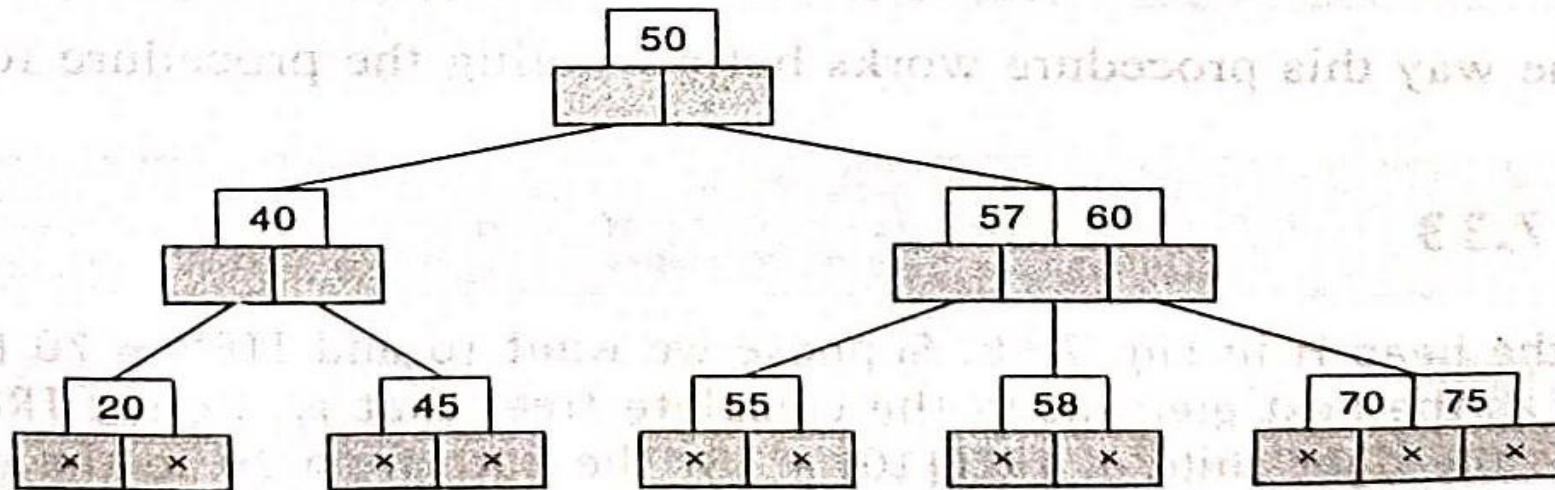
Initial Tree



Delete 35:



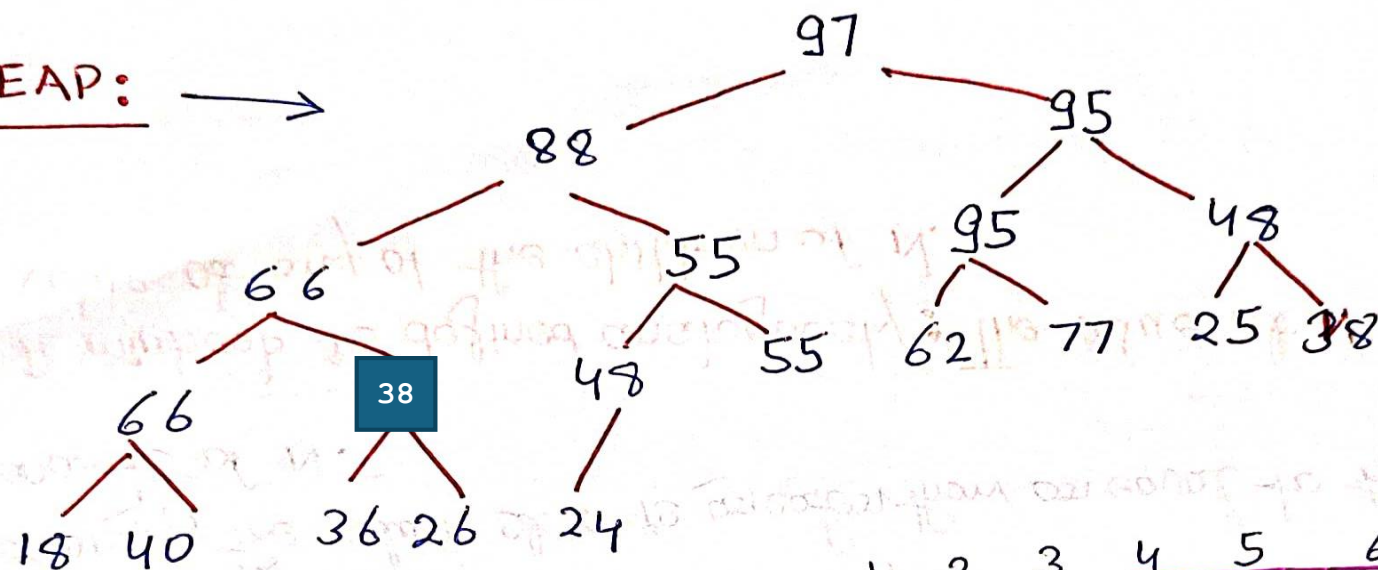
Delete 65:



HEAP: HEAPSORT:-

- The heap is used in an elegant sorting algorithm called **heapsort**.
- Suppose (H) is a complete binary tree with n elements.
- Unless otherwise stated, we assume that H is maintained in memory by a linear array **TREE** using the sequential representation of H not linked representation.
- H is called a heap or a maxheap, if each node N of H has following properties:
The value at N is greater than or equal to the value of the children of N .
- Accordingly, the value at N is greater than or equal to the value of any of the descendants of N .
- ** A minheap is defined analogously: The value at N is less than or equal to the value of any of the children of N .

HEAP: →



Sequential representation: TREE

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
97	88	95	66	55	95	48	66	38	48	55	62	77	25	38
16	17	18	19	20										

Inserting into a Heap:

Suppose H is a heap with N elements, and suppose an ITEM of information is given. We insert ITEM into the heap H as follows

- ① First adjoin ITEM at the end of H so that H is still a complete tree, but not necessarily a heap
- ② Then let ITEM rise to its appropriate place in H so that H is finally a heap.

Procedure :- $\text{INHEAP}(\text{TREE}, N, \text{ITEM})$

- > A heap H with N elements is stored in the array TREE and an ITEM of information is given
- > This procedure inserts ITEM as a new element of H
- > PTR gives the location of ITEM as it rises in the tree
- > PAR denotes the location of the parent of ITEM

1. > [Add new node to H and initialize PTR .]

Set $N : N + 1$ and $\text{PTR} = N$

2. > [Find location to insert ITEM] 3. > Set $\text{PAR} = \lfloor \text{PTR} / 2 \rfloor$

4. > Repeat steps 3 to 6 while $\text{PTR} < 1$

If $\text{ITEM} \leq \text{TREE}[\text{PAR}]$, then:

Set $\text{TREE}[\text{PTR}] = \text{ITEM}$ and Return

5. > Set $\text{TREE}[\text{PTR}] = \text{TREE}[\text{PAR}]$ /* Moves node down */

6. > Set $\text{PTR} = \text{PAR}$ /* Updates PTR */

7. > [Assign ITEM as the root of H]
Set $\text{TREE}[1] = \text{ITEM}$

8. > Return

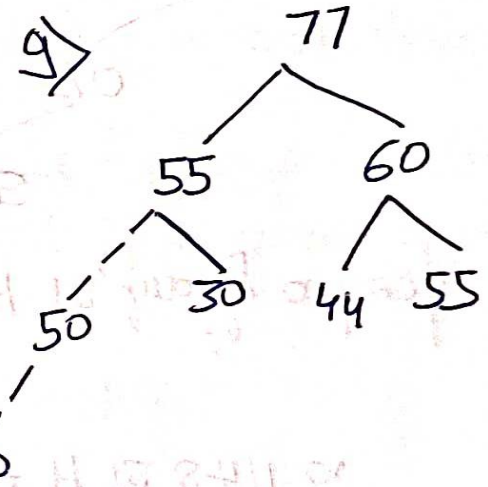
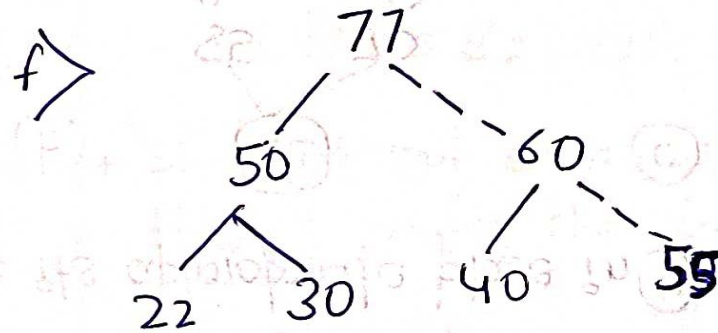
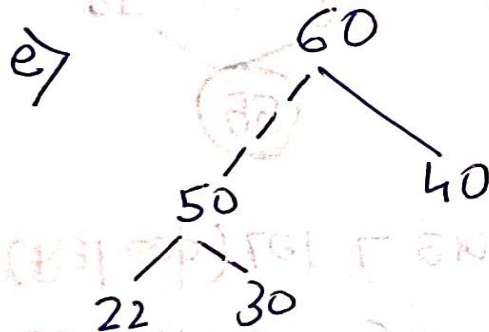
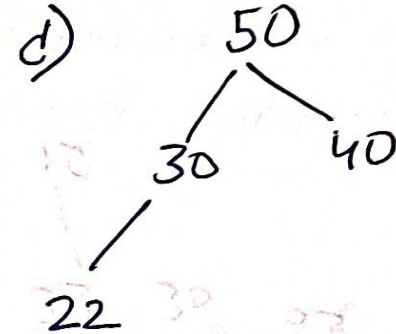
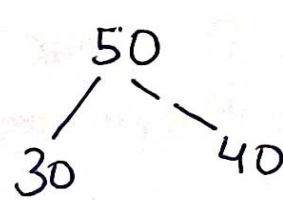
Building heap (H) from the list of element :-

44, 30, 50, 22, 60, 55, 77, 55

a) 44

b) 44

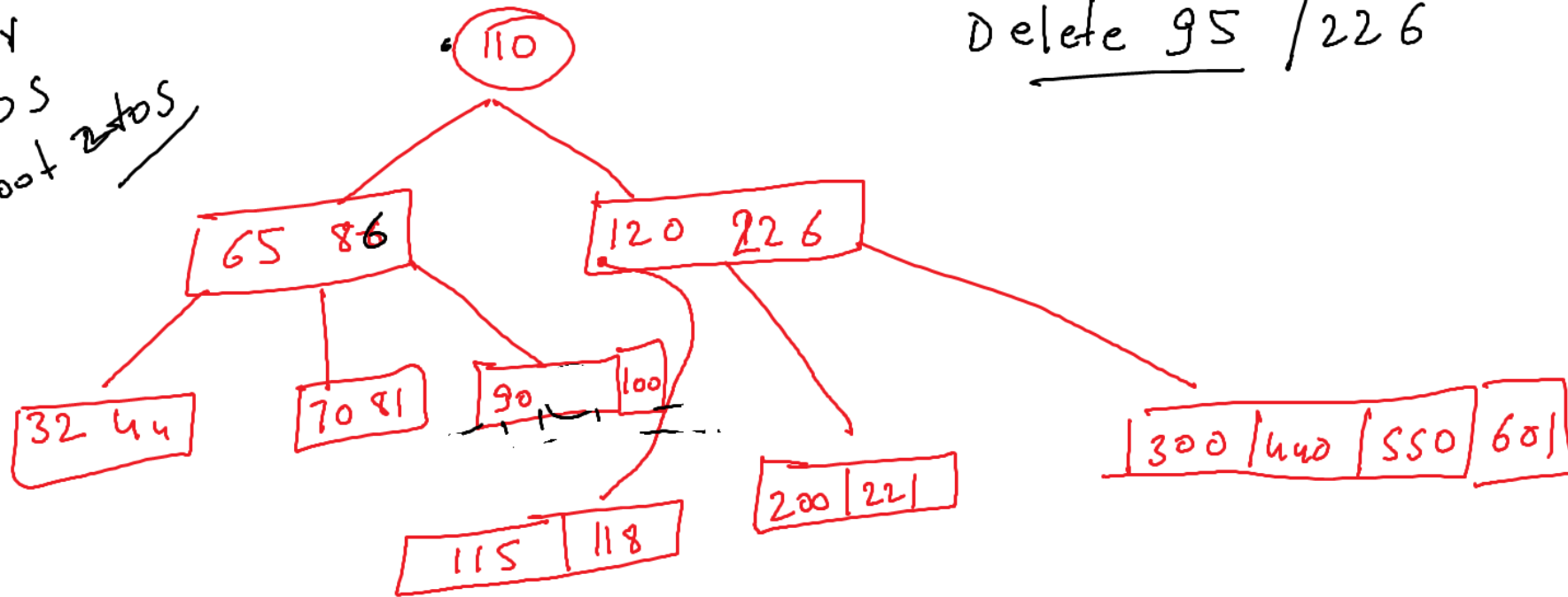
c)



* Building a Heap

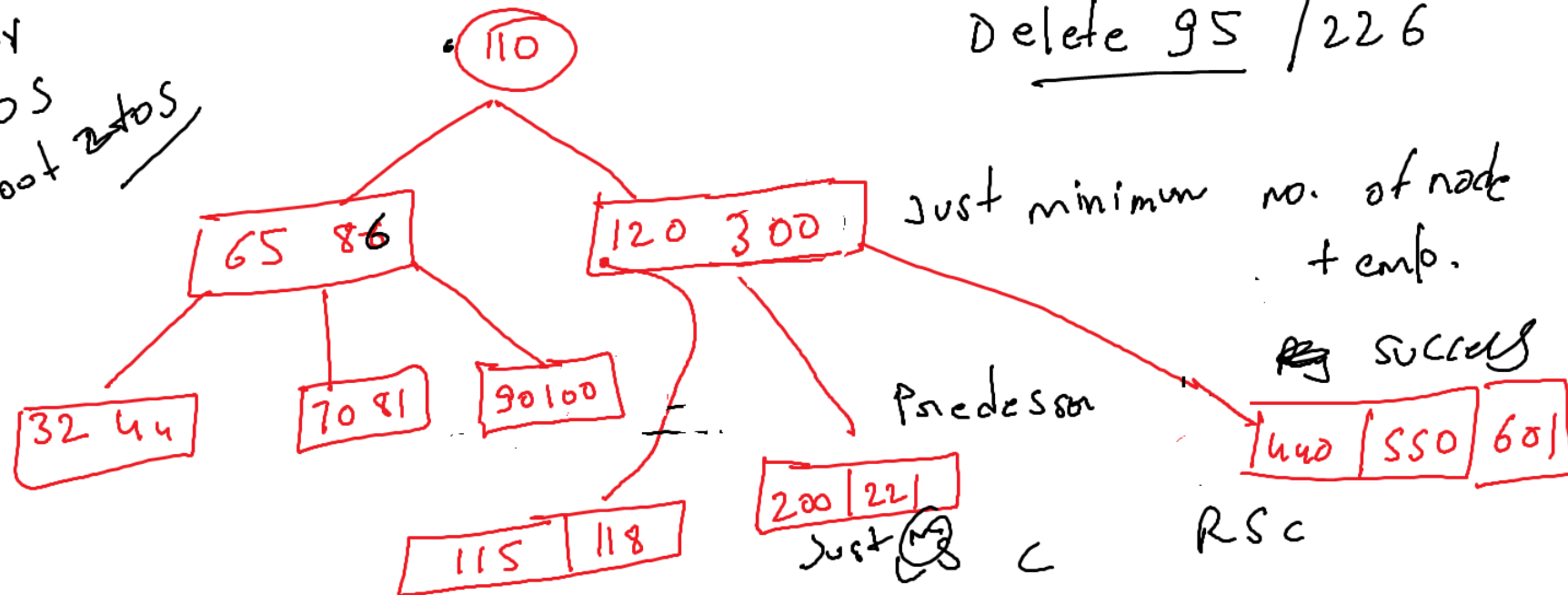
5-way
3 to 5
root 2 to 5

Delete 95 / 226



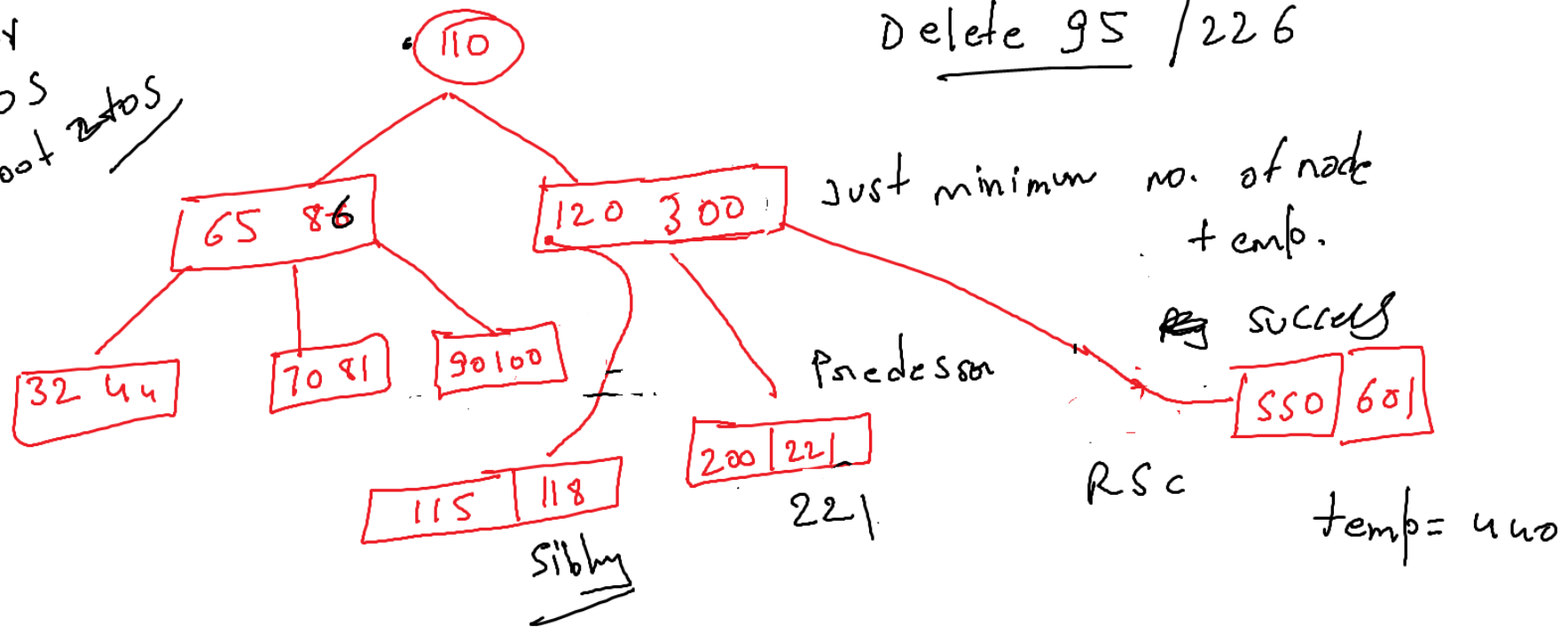
5-way
3 to 5
root 2 to 5

Delete 95 / 226

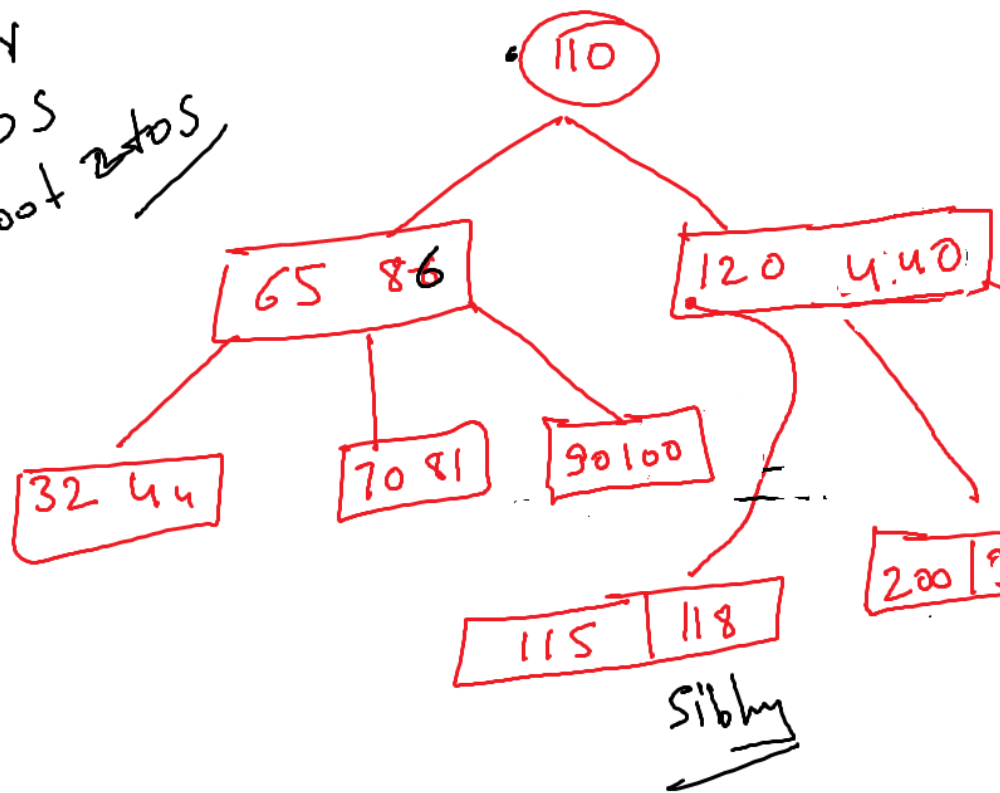


5-way
3 to 5
root 2 to 5

Delete 95 / 226



5-way
3 to 5
500 to 2005



Delete 221

Just minimum no. of node
+ emb.

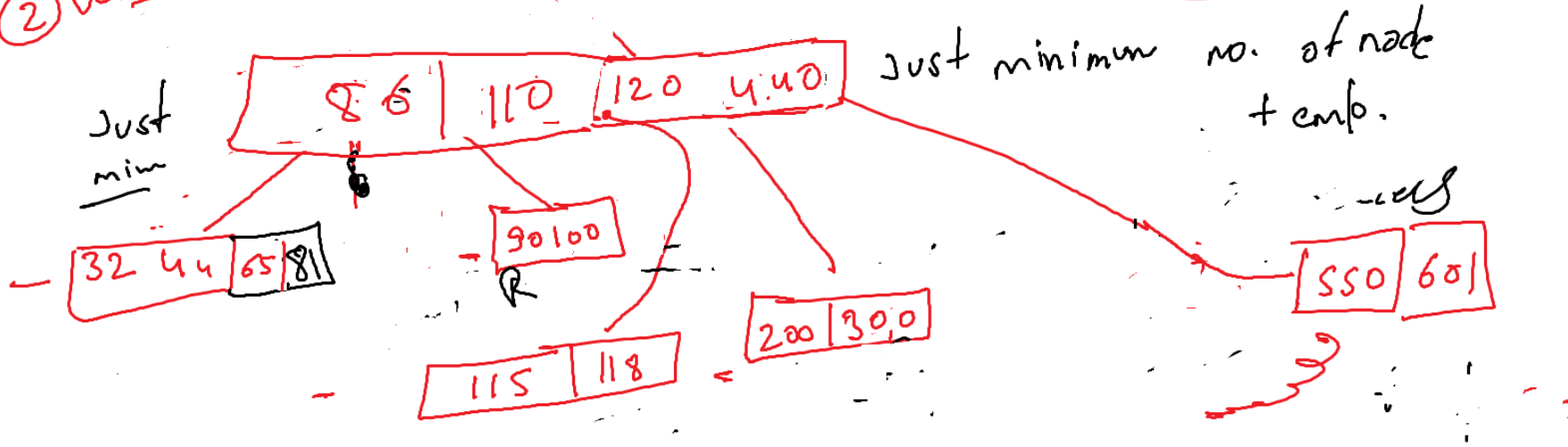
Predecessor

Successor

RSC

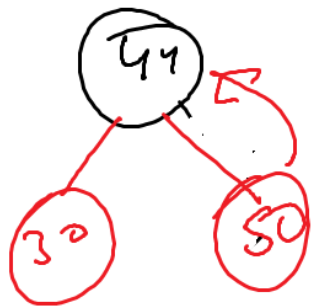
② violation

Delete (70)



44, 30, 50, 22, 60, 55, 77, 55

We have to follow the principle of complete binary tree.



heapify

